OS ASSIGNMENT 4

SAHANA K S
PES2UG24CS425
SECTION G

Phase 1 : Object Storage Foundation

1A :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

1B :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2 : Tree Objects

2A :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ make test_tree 2>/dev/null
./test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9
db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

Phase 3 : The Index (Staging Area)

3A :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ ./pes init
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Added: file1.txt
Added: file2.txt
Staged changes:
  new file:   file1.txt
  new file:   file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  README.md
  pes
  test_objects.c
  test_objects.o
  Makefile
  test_tree.o
  pes.h
  .gitignore
  tree.c
  pes.c
  commit.h
  test.txt
  object.o
  index.h
  index.c
  test_sequence.sh
  tree.h
```

3B :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ cat .pes/index
100664 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776762724 12 file1.txt
100664 0c45965bdd587e7005375a69f67cad4e61d9357c65f015e0d569c15165f433b1 1776762724 16 file2.txt
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

Phase 4 : Commits and History

4A :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ ./pes log
commit fd64cd76795f6afbb251ed518a8423d11a9a89d79523b93913b6ab76a9c29123
Author: PES User <pes@localhost>
Date:   2026-04-21 11:00:14

    Add farewelldd f

commit fbe57ce36609944d80c22b29a869b81746528b2e8abfc30fa977bf379de2baed
Author: PES User <pes@localhost>
Date:   2026-04-21 10:58:57

    Add worldelldd f!

commit 97dffde59d19154f5fe8f6afe29ca6436bc920a4c4660d600cd137cd6ce7f48d
Author: PES User <pes@localhost>
Date:   2026-04-21 10:58:33

    Initial commit

sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

4B :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/47/3a0f4c3be8a93681a267e3b1e9a7dcda1185436fe141f7749120a303721813
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/8c/91b0ec0d2af43e437cfada4e66ba7bf1ee9d50898d6e712586ef7c6e6c0a5a
.pes/objects/97/dffde59d19154f5fe8f6afe29ca6436bc920a4c4660d600cd137cd6ce7f48d
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/fb/e57ce36609944d80c22b29a869b81746528b2e8abfc30fa977bf379de2baed
.pes/objects/fd/64cd76795f6afbb251ed518a8423d11a9a89d79523b93913b6ab76a9c29123
.pes/refs/heads/main
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

4C :

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
fd64cd76795f6afbb251ed518a8423d11a9a89d79523b93913b6ab76a9c29123
ref: refs/heads/main
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

<u>Final : Full Integration Test</u>

```
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
test_sequence.sh: line 26: ./pes: No such file or directory
make: *** [Makefile:45: test-integration] Error 127
sahana_pes2ug24cs425@ubuntu:~/PES2UG24CS425-pes-vcs$
```

ANALYSIS QUESTIONS

Q1.1: Sharding distributes files across many directories so each directory has few entries, avoiding slowdown from large directory lookups and inode management issues in filesystems when thousands of files exist in a single folder.

Q1.2: Writing directly risks partial or corrupted objects if a crash occurs mid-write, rename() is atomic in POSIX so the file either appears fully written or not at all ensuring consistency.

Q1.3: Using birthday paradox, collision ≈ $n^2/(2 \cdot 2^b)$; for 10^9 objects SHA-1 ≈ $10^{18} / 2^{161} \approx 10^{-30}$, SHA-256 ≈ $10^{18} / 2^{257} \approx 10^{-60}$; Git is migrating because SHA-1 is cryptographically broken and vulnerable to deliberate collisions while SHA-256 is stronger.

Q2.1: Sorting ensures deterministic tree hashes so identical directory structures produce identical tree objects, otherwise different filesystem orders would yield different hashes and break consistency across developers.

Q2.2: The executable bit affects program behaviour so tracking it preserves whether a file should run as a script/binary, without it code may fail to execute or behave incorrectly across systems.

Q2.3: Trees only store entries mapping names to object IDs, so without files there is nothing to reference and no object created, making empty directories impossible to represent in this model.

Q3.1: mtime and size let pes status quickly detect unchanged files without rehashing contents, but worst case is a file modified while keeping same size and timestamp (e.g., rapid edits or manual timestamp reset) so changes go undetected.

Q3.2: Sorting ensures deterministic ordering so the same index state produces identical file content and consistent behavior, otherwise different orders would cause unnecessary differences and inconsistencies.

Q3.3: write-temp-then-rename ensures the index is always either old or fully new, if written directly and a crash occurs the index could become partially written or corrupted leaving the repository in an inconsistent state.

Q4.1: The ref is the pointer to the current branch tip, so updating it makes the new commit reachable from HEAD and part of history, without this the commit exists but is not referenced.

Q4.2: The commit object exists but is unreachable (dangling), it is still recoverable by scanning objects since objects are immutable and safely written before ref update.

Q4.3: A partial overwrite would leave a truncated or invalid hash causing the branch to point to a non-existent object and effectively corrupting the repository state.

Q5.1: Update .pes/HEAD to point to refs/heads/<branch> and ensure the branch file exists with its commit hash, read that commit's tree and update the working directory to match it while updating the index, it is complex due to handling uncommitted changes, conflicts, and keeping HEAD, index, and working directory consistent.

Q5.2: Compare each tracked file's current working directory hash with the index hash to detect local modifications, then compare the target branch's tree hash for that path, if the file is modified locally and differs between branches refuse checkout.

Q5.3: Commits made in detached HEAD are created but not referenced by any branch so they become dangling, they can be recovered by creating a new branch pointing to that commit hash before garbage collection removes them.

Q6.1: Start from all branch heads, traverse commits → trees → blobs marking each visited hash as reachable using a hash set, then scan .pes/objects and delete any not in the set, with 50 branches and 100k commits you visit ~100k commits plus associated trees and blobs (typically a few hundred thousand objects total).

Q6.2: GC could delete an object written by a commit before the ref is updated, making the new commit point to a missing object, Git avoids this using reachability checks with safety windows and by not deleting recent objects or running GC concurrently with active writes.

GITHUB LINK

https://github.com/sahanaks1011/PES2UG24CS425-pes-vcs